Proper Notes Format

Perfect. We'll continue in the **same deep-dive note style**.

◆ EMAILSERVICE CLASS

```
@Service
public class EmailService {
```

This is your Service Layer.

Remember the project flow:

```
User
  ↓
Controller
  ↓
Service
  ↓
Flask
  ↓
ML Model
```

The controller receives the request.

The service performs the actual work.

Why Do We Need A Service Layer?

Suppose we put everything inside the controller.

```
@PostMapping("/analyze")
public ResponseEntity<EmailResponseDTO> analyze(...) {

    // Create headers
    // Call Flask
    // Process response
    // Handle errors
    // Build DTO

}
```

The controller becomes huge and difficult to maintain.

Instead:

```
Controller
```

handles requests.

```
Service
```

handles business logic.

Restaurant Analogy

```
Customer
   ↓
Waiter
   ↓
Chef
```

Customer:

```
User
```

Waiter:

```
EmailController
```

Chef:

```
EmailService
```

The waiter does not cook food.

The chef does.

Similarly:

Controller does not analyze emails.

Service does.

◆ @Service

```
@Service
```

This annotation tells Spring:

> "This class is a Service Bean."

When Spring starts:

```
@SpringBootApplication
```

it scans the project.

When it finds:

```
@Service
```

Spring says:

> "Create and manage an EmailService object."

Internally something like:

```
EmailService emailService =
    new EmailService();
```

is created.

Spring stores it inside its container.

```
Spring Container
 ├── EmailController
 ├── EmailService
 └── Other Beans
```

Because Spring manages it, the controller can receive it automatically through constructor injection.

◆ @Value("${flask.api.url}")

```
@Value("${flask.api.url}")
private String flaskUrl;
```

This line confuses many beginners.

Let's go slowly.

What is flaskUrl?

```
private String flaskUrl;
```

A String variable.

Example:

```
String name;
```

stores a name.

Similarly:

```
String flaskUrl;
```

stores a URL.

Example Value

Suppose application.properties contains:

```
flask.api.url=http://localhost:5000/predict
```

Spring reads this property.

Then automatically assigns:

```
flaskUrl =
"http://localhost:5000/predict";
```

What Does @Value Mean?

```
@Value("${flask.api.url}")
```

means:

> "Take the value of flask.api.url from application.properties and put it into this variable."

Think of it as:

```
application.properties
        ↓
flaskUrl variable
```

Why Is This Useful?

Bad approach:

```
String flaskUrl =
"http://localhost:5000/predict";
```

Hardcoded.

If URL changes:

```
http://localhost:6000/predict
```

you must modify source code.

Better approach:

```
flask.api.url=http://localhost:5000/predict
```

Change only configuration.

No code changes.

◆ RESTTEMPLATE

```
private final RestTemplate restTemplate =
        new RestTemplate();
```

This object is extremely important.

What Is RestTemplate?

RestTemplate is a Spring class used to call external APIs.

Your application talks to:

```
Flask
```

using HTTP requests.

Remember:

```
Swagger
   ↓
Spring
```

Swagger sends HTTP requests to Spring.

Similarly:

```
Spring
   ↓
Flask
```

Spring sends HTTP requests to Flask.

RestTemplate performs this communication.

Think Of It As A Messenger

```
Spring
   ↓
Messenger
   ↓
Flask
```

Messenger:

```
RestTemplate
```

Without RestTemplate:

Spring cannot easily send HTTP requests.

Why final?

```
private final RestTemplate restTemplate
```

means:

Once created:

```
new RestTemplate()
```

cannot be replaced.

Makes the object safer and more predictable.

◆ ANALYZE METHOD

```
public EmailResponseDTO analyze(String content) {
```

This is the main method inside EmailService.

public

```
public
```

means:

Other classes can call this method.

Example:

```
emailService.analyze(...)
```

inside EmailController.

EmailResponseDTO

```
EmailResponseDTO
```

Return type.

Means:

> "This method returns an EmailResponseDTO object."

Example:

```
new EmailResponseDTO(
    "SPAM",
    1.0
)
```

analyze

```
analyze
```

Method name.

String content

```
String content
```

Parameter.

Stores incoming email text.

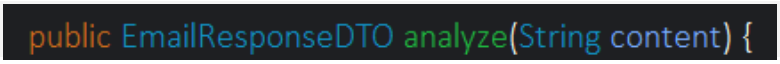
Example:

Controller calls:

```
emailService.analyze(
    "You won a lottery"
);
```

Then:

```
content
```



contains:

```
You won a lottery
```

◆ TRY-CATCH BLOCK

```
try {
```

and

```
catch (Exception e) {
```

Why Use Try-Catch?

Many things can fail.

Examples:

```
Flask not running

Wrong URL

Network issue

Timeout

Server error
```

Without try-catch:

Application crashes.

With try-catch:

Application handles errors gracefully.

Flow

```
Try
 ↓
Success?
 ↓
Yes → Continue

No
 ↓
Catch Block
```

◆ HTTPHEADERS

```
HttpHeaders headers =
        new HttpHeaders();
```

Creates an HTTP headers object.

What Are Headers?

Every HTTP request contains:

```
Headers
+
Body
```

Example:

```
POST /predict
```

Headers:

```
Content-Type: application/json
```

Body:

```
{
   "emailText":"You won a lottery"
}
```

Headers provide extra information about the request.

◆ SET CONTENT TYPE

```
headers.setContentType(
        MediaType.APPLICATION_JSON
);
```

This tells Flask:

> "The data I am sending is JSON."

Result:

```
Content-Type: application/json
```

Without this header:

Flask may not correctly understand the request body.

◆ HTTPENTITY

```
HttpEntity<Map<String,String>> entity =
        new HttpEntity<>(
                Map.of(
                    "emailText",
                    content
                ),
                headers
        );
```

```
HttpEntity<Map<String, String>> entity =
        new HttpEntity<>(Map.of("emailText", content), headers);
```

This is one of the most important lines.

What Is HttpEntity?

HttpEntity represents a complete HTTP request.

Contains:

```
Headers
+
Body
```

Think of it as a package.

```
Package
 ├── Headers
 └── Body
```

Body Creation

```
Map.of(
   "emailText",
   content
)
```

creates:

```
{
   "emailText":"You won a lottery"
}
```

assuming content contains:

```
You won a lottery
```

Headers Added

```
headers
```

contains:

```
Content-Type:
application/json
```

Final Request:

```
Headers:
Content-Type: application/json

Body:
{
   "emailText":"You won a lottery"
}
```

This is exactly what gets sent to Flask.

Next section should continue with:

```
ResponseEntity<Map> response =
        restTemplate.postForEntity(
                flaskUrl,
                entity,
                Map.class
        );
```

because this is where Spring actually sends the request to Flask and waits for the prediction response.